Practical No. 6

Mapped CO: CO3 Units V Bloom Level: L6 – Create

1.0) AIM

To build a complete DApp with front-end interface that interact with a deployed smart contract.

2.0) SCOPE

This practical covers:

- Building a web-based DApp interface (HTML5, CSS3, JavaScript)
- Connecting MetaMask wallet
- Using Web3.js to interact with smart contracts
- Read operations (.call) vs Write operations (.send)
- Displaying real-time transaction results

3.0) FACILITIES / APPARATUS

Software Requirements:

- Ganache v2.7.1 - Local blockchain emulator (running on 127.0.0.1:7545)
- MetaMask v11+ - Browser wallet extension (Chrome/Firefox/Edge)
- Remix IDE (https://remix.ethereum.org) - For smart contract deployment
- Python 3.x - For running local HTTP web server
- Web browser - Chrome, Firefox, or Edge (latest version)
- Text editor - VS Code, Sublime Text, or any code editor

Libraries & Technologies:

- Web3.js v1.10.0 (loaded from CDN - https://cdn.jsdelivr.net/npm/web3@1.10.0)
- HTML5 for semantic structure
- CSS3 for professional styling and responsive design
- JavaScript ES6 for interactive logic and blockchain communication

4.0) THEORY

4.1 What is a DApp (Decentralized Application)?

A Decentralized Application (DApp) is a software application that combines three components:

• Frontend: Web interface (HTML/CSS/JavaScript) that users interact with

• Backend: Smart contract running on blockchain (immutable, transparent)
• Wallet: MetaMask or similar wallet for user identity and transaction signing

DApp = Frontend (HTML/CSS/JS) + Backend (Smart Contract) + Wallet (MetaMask)

Unlike traditional apps where data is centralized, DApps store data on blockchain, making them censorship-resistant and transparent.

4.2 The Three Components of This Practical

Component 1: HTML (index.html) - Structure

- Defines the layout and elements users see. Creates buttons, input fields, display areas. Like the skeleton of the application.

Component 2: JavaScript (app.js) - Logic

- Contains the "brain" of the DApp. Listens for clicks, communicates with blockchain via Web3.js, updates the interface based on results.

Component 3: CSS (styles.css) - Presentation

- Makes the DApp beautiful and professional. Adds colors, fonts, spacing, hover effects. The "skin" of the application.

4.3 How Web3.js Works

Web3.js is a JavaScript library that acts as a bridge between your browser and the blockchain:

Browser (Web3.js) ←→ Ganache Blockchain ←→ Smart Contract

Three Main Functions:

1. Connect to Blockchain:
   web3 = new Web3(window.ethereum);
   This creates a connection from browser to Ganache via MetaMask.

2. Create Contract Connection:
   contract = new web3.eth.Contract(ABI, contractAddress);
   This tells Web3 where the contract is and what functions it has.

3. Call Contract Functions:
   contract.methods.get().call(); // Read data (free)
   contract.methods.set(value).send(); // Write data (costs gas)
   This executes the contract functions.

4.4 MetaMask: The User's Wallet

MetaMask is a browser wallet that:

• Stores your private key (secret password for your account)
• Signs transactions (approves them with your private key)
• Manages multiple accounts (you can switch between them)
• Connects to different blockchains (Ganache, Ethereum, Polygon, etc.)

When you click "Confirm" in MetaMask:

1. MetaMask uses your private key to sign the transaction
2. The signed transaction is sent to the blockchain
3. The smart contract executes the transaction
4. New block is created with your transaction

4.5 Call vs Send - The Key Difference

Operation	Purpose	Gas Cost	Speed	Confirmation
.call()	Read data from blockchain	FREE (0 gas)	Instant	None needed
get()	Returns stored value	0 ETH	< 1 second	No MetaMask popup
.send()	Write/change data	PAID (⛽ gas)	2-5 seconds	MetaMask confirmation
set()	Stores new value	0.0001-0.001 ETH	3-5 seconds	MetaMask popup appears

4.6 Transaction Flow in a DApp

When a user sets a value, here's what happens:

1. User enters value in input field
2. User clicks "Set Value" button
3. JavaScript catches the click event
4. Web3.js creates a transaction with the new value
5. MetaMask popup appears asking to confirm
6. User clicks "Confirm" in MetaMask
7. MetaMask signs transaction with user's private key
8. Signed transaction sent to Ganache blockchain
9. Smart contract executes the set() function
10. New block created on blockchain
11. Transaction results displayed on DApp screen
    - Transaction hash (unique ID)
    - Gas used (blockchain processing cost)
    - Block number (which block contains this tx)
12. DApp automatically reads the new value

13. Screen updates to show the new value

This entire process takes 3-5 seconds and is permanent!

5.0) SMART CONTRACT USED

The SimpleStorage smart contract (same as Practical 5):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract SimpleStorage {
    uint256 private number;

    // Write function - stores a value (costs gas)
    function set(uint256 newNumber) public {
        number = newNumber;
    }

    // Read function - retrieves the value (free)
    function get() public view returns (uint256) {
        return number;
    }
}
```

Two functions:

- set(uint256): Changes the stored value (costs gas, requires blockchain transaction)
- get(): Returns the stored value (free, just reads from blockchain)

The value persists permanently on the blockchain even after you close the browser!

6.0) THE THREE FILES

FILE 1: index.html

What it does: Creates the web interface - buttons, input fields, display areas that user interact with.

Save as: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Practical 6: Web3 DApp with MetaMask</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <div class="container">
        <h1>Practical 6: Web3 DApp with MetaMask</h1>
        <p class="subtitle">Blockchain Fundamentals - Smart Contract
Interaction</p>

        <div class="section">
            <h2>Wallet Connection</h2>
            <button id="connectBtn" class="btn btn-primary">Connect
MetaMask Wallet</button>
            <div id="walletStatus" class="status-box">
                <p><strong>Status:</strong> <span id="statusText">Not
Connected</span></p>
                <p><strong>Account:</strong> <span
id="accountText">N/A</span></p>
                <p><strong>Network:</strong> <span
id="networkText">N/A</span></p>
            </div>
        </div>

        <div class="section">
            <h2>Contract Information</h2>
            <div class="info-box">
                <p><strong>Contract Address:</strong> <span
id="contractAddress">Loading...</span></p>
                <p><strong>Contract Status:</strong> <span
id="contractStatus">Not Deployed</span></p>
            </div>
        </div>

        <div class="section">
            <h2>Read Value (Free - No Gas)</h2>
            <button id="getBtn" class="btn btn-secondary">Get Current
Value</button>
            <div id="getResult" class="result-box" style="display:none;">
                <p><strong>Current Value:</strong> <span
```

```
id="currentValue">-</span></p>
            </div>
        </div>

        <div class="section">
            <h2>Set Value (Write - Costs Gas)</h2>
            <div class="input-group">
                <label for="valueInput">Enter New Value:</label>
                <input type="number" id="valueInput" placeholder="Enter a
number" min="0">
            </div>
            <button id="setBtn" class="btn btn-primary">Set
Value</button>
            <div id="setResult" class="result-box" style="display:none;">
                <p><strong>Transaction Hash:</strong> <span id="txHash">-
</span></p>
                <p><strong>Gas Used:</strong> <span id="gasUsed">-
</span></p>
                <p><strong>Block Number:</strong> <span
id="blockNumber">-</span></p>
                <p><strong>Status:</strong> <span
id="txStatus">Pending...</span></p>
            </div>
        </div>

        <div class="section">
            <h2>Transaction Log</h2>
            <div id="txLog" class="log-box">
                <p class="log-entry">No transactions yet...</p>
            </div>
        </div>

        <div id="errorBox" class="error-box" style="display:none;">
            <p><strong>Error:</strong> <span id="errorText"></span></p>
        </div>
    </div>

    <script
src="https://cdn.jsdelivr.net/npm/web3@1.10.0/dist/web3.min.js"></script>
    <script src="app.js"></script>
</body>
</html>
```

FILE 2: app.js

What it does: The "brain" of the DApp. Listens for clicks, communicates with blockchain, updates the screen.

Save as: app.js

IMPORTANT: Replace the contract address on line 5 with YOUR deployed contract address!

```
// PRACTICAL 6: WEB3 DAPP WITH METAMASK
// The "brain" of our DApp - handles all the logic

// ===== STEP 1: SET UP CONTRACT DETAILS =====
// REPLACE THIS with your deployed contract address
const contractAddress = '0x660db882172C119c13C93E9696E748b8FAb50e45';

// This tells Web3 what functions our contract has
const contractABI = [
  {
    "inputs": [{"internalType": "uint256", "name": "newNumber", "type":
"uint256"}],
    "name": "set",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "get",
    "outputs": [{"internalType": "uint256", "name": "", "type": "uint256"}],
    "stateMutability": "view",
    "type": "function"
  }
];

// ===== STEP 2: GLOBAL VARIABLES =====
let web3;           // Connection to blockchain
let contract;       // Connection to smart contract
let userAccount;    // User's MetaMask account
let isConnected = false; // Connected to MetaMask?

// ===== WHEN PAGE LOADS =====
window.addEventListener('load', async () => {
  console.log('Page loaded. Checking for MetaMask...');

  if (typeof window.ethereum !== 'undefined') {
    console.log('✅ MetaMask found!');
    web3 = new Web3(window.ethereum);
    contract = new web3.eth.Contract(contractABI, contractAddress);
    document.getElementById('contractAddress').textContent = contractAddress;
    document.getElementById('contractStatus').textContent = '✅ Deployed';
  } else {
    showError('MetaMask not installed!');
  }
});

// ===== WHEN USER CLICKS "CONNECT METAMASK WALLET" =====
```

```
document.getElementById('connectBtn').addEventListener('click', async () => {
  try {
    const accounts = await window.ethereum.request({
      method: 'eth_requestAccounts'
    });
    userAccount = accounts[0];
    console.log('✅ Connected to:', userAccount);
    updateAccountDisplay();
    enableAllButtons();
    isConnected = true;
    clearError();
  } catch (error) {
    showError('Failed to connect: ' + error.message);
  }
});

// ===== UPDATE SCREEN WITH ACCOUNT INFO =====
async function updateAccountDisplay() {
  try {
    document.getElementById('statusText').textContent = '✅ Connected';
    document.getElementById('accountText').textContent = userAccount;
    document.getElementById('networkText').textContent = '✅ Ganache (Local)';
    addLog('✅ Connected to account: ' + userAccount.substring(0, 10) + '...');
  } catch (error) {
    console.error('Error:', error);
  }
}

// ===== WHEN USER CLICKS "GET CURRENT VALUE" (READ - FREE!) =====
document.getElementById('getBtn').addEventListener('click', async () => {
  if (!isConnected) {
    showError('Please connect MetaMask first');
    return;
  }

  try {
    console.log('Reading value from contract...');
    document.getElementById('getBtn').disabled = true;
    document.getElementById('getBtn').textContent = 'Reading...';

    // .call() means read-only - doesn't cost gas!
    const result = await contract.methods.get().call();

    console.log('✅ Value:', result);
    document.getElementById('currentValue').textContent = result.toString() + 'n';
    document.getElementById('getResult').style.display = 'block';
    addLog('✅ Read value: ' + result.toString());
    clearError();
  } catch (error) {
    console.error('Error:', error);
    showError('Failed to read value: ' + error.message);
  } finally {
    document.getElementById('getBtn').disabled = false;
    document.getElementById('getBtn').textContent = 'Get Current Value';
  }
});

// ===== WHEN USER CLICKS "SET VALUE" (WRITE - COSTS GAS) =====
document.getElementById('setBtn').addEventListener('click', async () => {
  if (!isConnected) {
    showError('Please connect MetaMask first');
    return;
```

```
  }

  const valueInput = document.getElementById('valueInput').value;
  if (!valueInput || valueInput < 0) {
    showError('Please enter a valid number');
    return;
  }

  try {
    console.log('Setting value to:', valueInput);
    document.getElementById('setBtn').disabled = true;
    document.getElementById('setBtn').textContent = 'Sending...';

    // Estimate gas needed
    const gasEstimate = await contract.methods.set(valueInput).estimateGas({
      from: userAccount
    });

    // .send() means write to blockchain - costs gas!
    // MetaMask popup will appear asking to confirm
    const tx = await contract.methods.set(valueInput).send({
      from: userAccount,
      gas: gasEstimate.toString()
    });

    console.log('✅ Transaction confirmed!', tx);

    // Show results on screen
    document.getElementById('txHash').textContent = tx.transactionHash;
    document.getElementById('gasUsed').textContent = tx.gasUsed.toString() + '
units';
    document.getElementById('blockNumber').textContent = tx.blockNumber.toString();
    document.getElementById('txStatus').textContent = '✅ Confirmed';
    document.getElementById('setResult').style.display = 'block';

    document.getElementById('valueInput').value = '';
    addLog('✅ Set value to ' + valueInput + ' (Gas: ' + tx.gasUsed.toString() +
')');

    // Automatically read new value
    setTimeout(() => {
      document.getElementById('getBtn').click();
    }, 1000);

    clearError();
  } catch (error) {
    console.error('Error:', error);
    if (error.code === 4001) {
      showError('Transaction rejected by user');
    } else {
      showError('Failed: ' + error.message);
    }
  } finally {
    document.getElementById('setBtn').disabled = false;
    document.getElementById('setBtn').textContent = 'Set Value';
  }
});

// ===== HELPER FUNCTIONS =====
function enableAllButtons() {
  document.getElementById('getBtn').disabled = false;
  document.getElementById('setBtn').disabled = false;
```

```
}

function showError(message) {
  document.getElementById('errorBox').style.display = 'block';
  document.getElementById('errorText').textContent = message;
}

function clearError() {
  document.getElementById('errorBox').style.display = 'none';
}

function addLog(message) {
  const logBox = document.getElementById('txLog');
  const timestamp = new Date().toLocaleTimeString();

  if (logBox.innerHTML.includes('No transactions yet')) {
    logBox.innerHTML = '';
  }

  const logEntry = document.createElement('p');
  logEntry.className = 'log-entry';
  logEntry.textContent = '[' + timestamp + '] ' + message;
  logBox.insertBefore(logEntry, logBox.firstChild);
}

console.log('✅ App.js loaded!');
```

FILE 3: styles.css

What it does: Makes the DApp beautiful with colors, fonts, layout, and effects.

Save as: styles.css

This file doesn't need to be modified - just copy-paste as-is.

```
/* PRACTICAL 6: WEB3 DAPP STYLING */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    min-height: 100vh;
    padding: 20px;
    color: #333;
}

.container {
    max-width: 900px;
    margin: 0 auto;
    background: white;
    border-radius: 12px;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
    padding: 40px;
}

h1 {
    color: #667eea;
    text-align: center;
    margin-bottom: 10px;
    font-size: 2.5em;
}

.subtitle {
    text-align: center;
    color: #666;
    margin-bottom: 40px;
    font-size: 1.1em;
}

.section {
    background: #f8f9fa;
    padding: 25px;
    margin-bottom: 25px;
    border-radius: 8px;
    border-left: 4px solid #667eea;
}

.section h2 {
    color: #667eea;
    margin-bottom: 20px;
    font-size: 1.5em;
}
```

```
.btn {
    padding: 12px 30px;
    border: none;
    border-radius: 6px;
    font-size: 1em;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
    margin: 10px 0;
}

.btn:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}

.btn-primary {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
}

.btn-primary:hover:not(:disabled) {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

.btn-secondary {
    background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
    color: white;
}

.status-box, .info-box, .result-box {
    background: white;
    padding: 20px;
    border-radius: 6px;
    margin-top: 15px;
    border: 2px solid #e9ecef;
}

.status-box p, .info-box p, .result-box p {
    margin: 10px 0;
}

.input-group {
    margin-bottom: 15px;
}

.input-group label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
}

.input-group input {
    width: 100%;
    padding: 12px;
    border: 2px solid #e9ecef;
    border-radius: 6px;
    font-size: 1em;
}

.input-group input:focus {
```

```
    outline: none;
    border-color: #667eea;
}

.log-box {
    background: #f8f9fa;
    padding: 15px;
    border-radius: 6px;
    max-height: 300px;
    overflow-y: auto;
    border: 2px solid #e9ecef;
}

.log-entry {
    font-size: 0.9em;
    padding: 8px;
    margin: 5px 0;
    background: white;
    border-radius: 4px;
    font-family: 'Courier New', monospace;
}

.error-box {
    background: #fff3cd;
    border: 2px solid #ffc107;
    border-radius: 6px;
    padding: 15px;
    margin: 20px 0;
    color: #856404;
}
```

7.0) PROCEDURE / STEPS TO EXECUTE

Step 1: Deploy Contract & Get Address (5 min)

Deploy SimpleStorage contract in Remix to Ganache. Copy the contract address. Keep this address for Step 3.**(NOTE:For deployment of smart contract follow the steps of practical 5)**

Step 2: Create Project Folder & Files (5 min)

Create a folder named "web3-dapp". Inside it, create 3 files and copy-paste the code from Section 6 above.
- index.html
- app.js
- styles.css

Step 3: Update Contract Address (3 min)

Open app.js file. On line 5, replace the placeholder with your actual deployed contract address from Step 1.

Step 4: Start Web Server (5 min)

Open Terminal/Command Prompt. Navigate to the "web3-dapp" folder. Run: python -m http.server 8000
You should see: "Serving HTTP on 0.0.0.0 port 8000"

Step 5: Prepare MetaMask (5 min)

Make sure Ganache is running. Make sure MetaMask is connected to Ganache Local network (Chain ID: 1337, RPC: 127.0.0.1:7545). Select an account with 100 ETH.

Step 6: Open DApp in Browser (3 min)

Open web browser. Go to: http://localhost:8000/index.html
You should see the DApp with purple gradient background and buttons.

Step 7: Test the DApp (10 min)

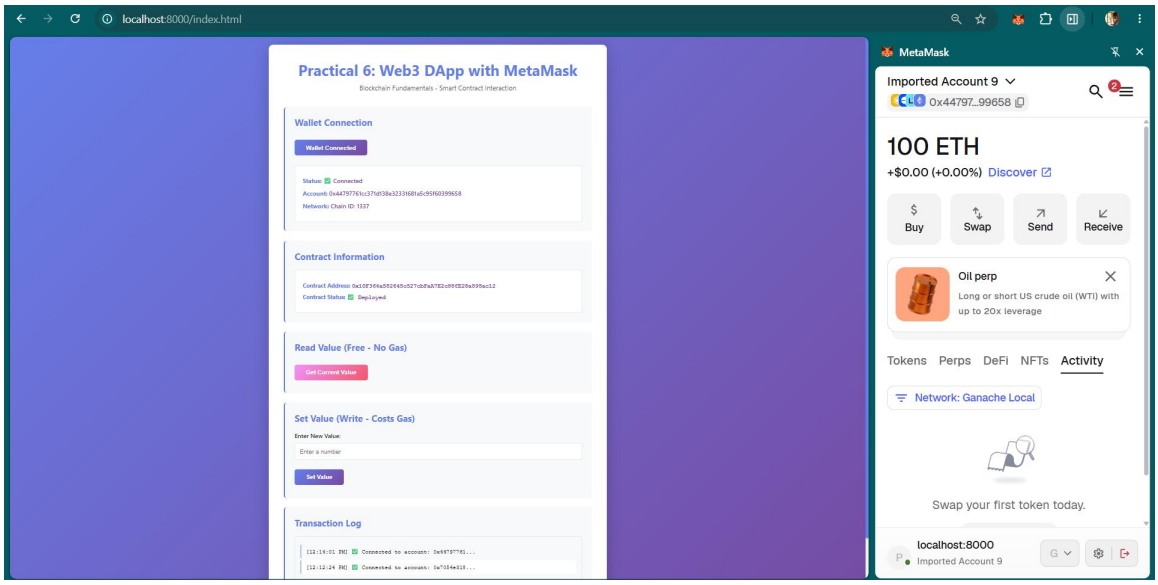
Follow the testing procedure in Section 8 below.

8.0) TESTING PROCEDURE

Test 1: Connect MetaMask Wallet

1. Click "Connect MetaMask Wallet" button
2. MetaMask popup appears - SELECT AN ACCOUNT WITH 100 ETH
3. Click "Connect" in MetaMask

4. Page should show:
   - Status: ✅ Connected
   - Account: Shows your account address
   - Network: ✅ Ganache (Local)



Test 2: Read Current Value (FREE - No Gas)

1. Click "Get Current Value" button
2. Wait 1-2 seconds
3. Result appears showing current value (probably 0)
4. NO MetaMask popup (reading is free)
5. Transaction log shows: "✅ Read value: 0"

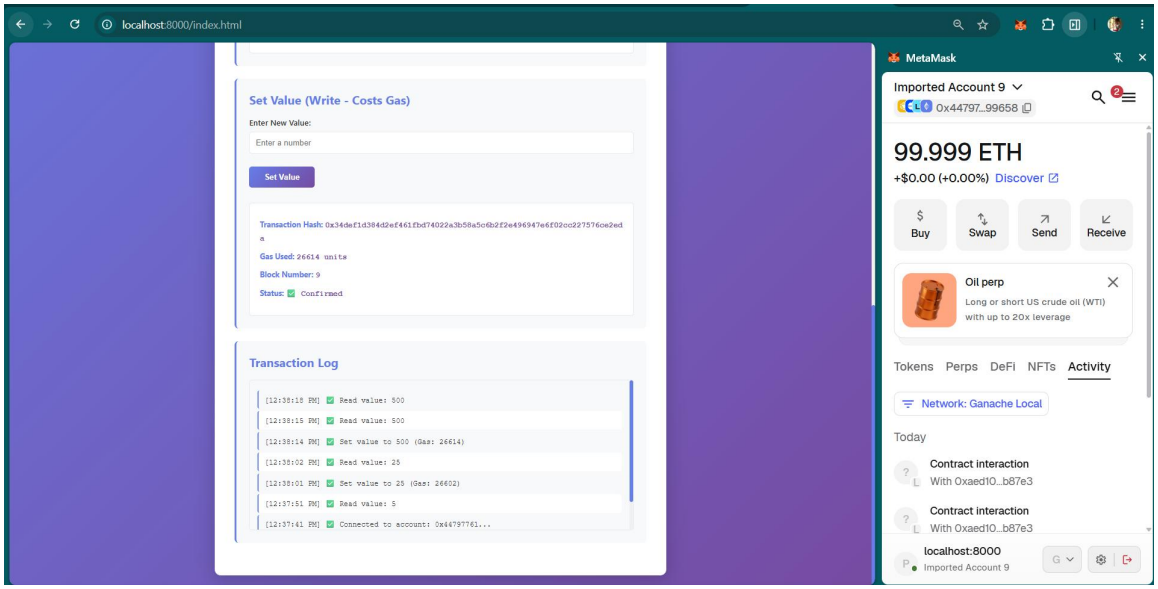
Test 3: Write New Value (COSTS GAS)

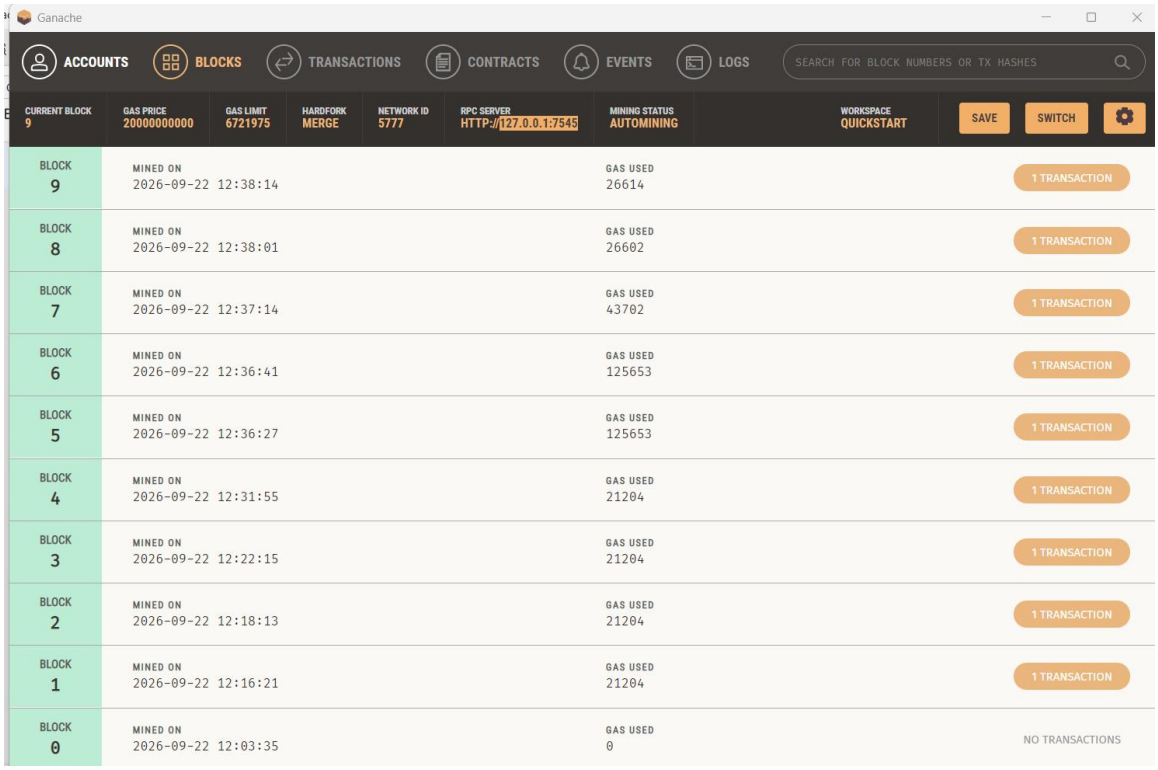
1. Enter a number in the input field (e.g., 999)
2. Click "Set Value" button
3. Button changes to "Sending..." (showing it's processing)
4. MetaMask popup appears to confirm transaction
5. Click "Confirm" in MetaMask
6. Wait 3-5 seconds for blockchain confirmation
7. Page shows:
   - Transaction Hash: 0x... (unique ID)
   - Gas Used: 43702 units (blockchain processing cost)
   - Block Number: 3 (which block stored this transaction)
   - Status: ✅ Confirmed
8. Transaction log shows: "✅ Set value to 999 (Gas: 43702)"

Test 4: Verify Value Changed (Permanent Storage)

1. Click "Get Current Value" button again
2. Should now show: 999 (the value you just set)
3. This proves the value is stored PERMANENTLY on blockchain
4. Even if you close browser and restart, value will still be 999
5. Transaction log shows: "✓ Read value: 999"

9.0) EXPECTED OUTPUT





10.0) CONCLUSION

Practical 6 successfully demonstrates complete DApp development using:

Frontend: Web interface with HTML5, CSS3, JavaScript

Blockchain: Web3.js integration with read (.call) and write (.send) operations

Security: MetaMask wallet integration and transaction signing

Architecture: Real-world DApp pattern used in production applications